Contents

From flood map to failure probability — the complete study guide **1**
How to use this guide . . . 2
0. The whole pipeline on one page . . . 2

PART I — THE WORLD WE BUILT **2**
1. The question . . . 2
2. The city as a graph . . . 3
3. The floods — where the maps come from . . . 4
4. Preprocessing — turning maps into per-site numbers . . . 5

PART II — THE SIMULATOR (WORLD 1) **5**
5. One Monte Carlo run, step by step . . . 5
6. From runs to targets . . . 7

PART III — THE FAST MODELS (WORLD 2) **8**
7. Why a surrogate, and how we score one . . . 8
8. The two-stage model (Weeks 19–31) . . . 8
9. The direct model (Week 32) — every step, with calculations . . . 9

PART IV — THE NEW APPROACH: MAKE IT END-TO-END **11**
10. What was asked, precisely . . . 11
11. The honest framing (say this first) . . . 11
12. The central problem: identifiability (understand this fully) . . . 11
13. Where the information about fragility actually is . . . 12
14. The fragility head — choices and the recommended one . . . 13
15. Your “stacking” idea — named properly, and the three ways to train it . . . 14
16. Leakage, and the two plugs . . . 14
17. Timing: keep it out of the fragility head . . . 15
18. The experiment that can actually show learning beats the formula: World 1b . . . 15
19. Further ahead: calibrating to Sandy . . . 15
20. Ensembles (the other meaning of your idea) . . . 16

PART V — THE PLAN **16**
21. Evaluation arms . . . 16
22. Metrics . . . 16
23. Draft predictions (to freeze before training, exactly like Week 32) . . . 17
24. Implementation map (when we build it) . . . 17

PART VI — DEFENDING IT **18**
25. Questions and answers . . . 18
26. Glossary . . . 19
27. Numbers to know . . . 19

From flood map to failure probability — the complete study guide

For Tanmay. Written 3 Nov 2026 from the repository (`feat/direct-model`, and the closed `feat/dynamic-flood-maps` tagged `week31-two-stage-results`), the Week 32 runs, and the research report on end-to-end fragility. Every number that comes from our data has been checked against the files. Where something is a design choice or an assumption, it says so.

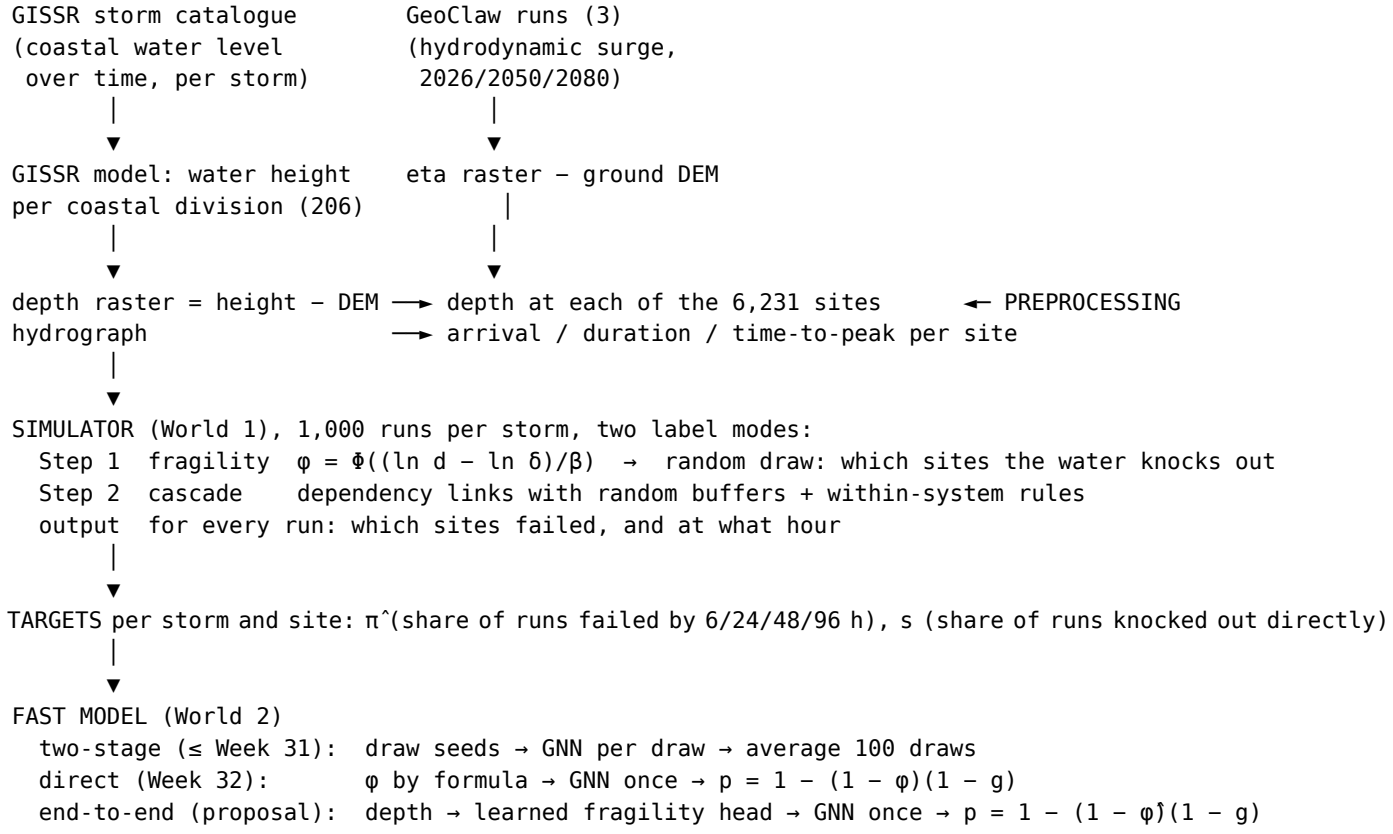
How to use this guide

The guide follows the order in which the information travels: **city → flood → simulator → targets → fast model → the end-to-end proposal**. Each chapter answers four questions: what it is, how it is computed, why it was done this way, and what you would say if challenged. Worked examples use real storms, mostly `syn_ts_914_6_1p2955` (the small old storm, 2.17 m) and `syn2_t198w16_wl4p722` (the largest, 4.72 m).

If you can redraw the diagram in §0 from memory and explain each arrow, you understand the project.

---

0. The whole pipeline on one page



PART I — THE WORLD WE BUILT

1. The question

When a storm surge floods New York, some infrastructure fails *directly*: water reaches a substation and trips it. Much more fails *indirectly*. A cell site loses power and runs on battery until the battery is empty. A hospital runs on generator until its fuel runs out. A subway section goes dark because its traction power is gone. We want, for every site in the city, **the probability that it has failed by 6, 24, 48 and 96 hours after the storm's peak**, together with how that depends on the flood.

The **horizons** are fixed at 6/24/48/96 h after the peak because the simulator is evaluated at those instants (Week 27 decision: horizons stay fixed and physical; finer grids change only the tipping share).

Why the cascade matters: in the October storms, total failures by 96 h are 2–4× the number of directly flooded sites. For 914_6, the expected number of seeds is 91, but failures by 96 h number 313.

2. The city as a graph

Sites (nodes): 6,231, frozen since the v1 campaigns (`data/graph_v1_frozen/`):

type	count	where the data came from (`src/graph/build_graph.py`)
power	203	HIFLD substations (208 in the source); those west of the Hudson are flagged external
telecom	4,150	20,190 cell towers grouped into ~500 m grid clusters
hospital	61	NYS health-facility list
subway	493	MTA stations
water	137	water and wastewater facilities
fuel	1,187	1,181 gas stations + petroleum terminals

External sites: 99. These are mostly New Jersey substations and petroleum terminals outside the city. They can receive a cascade but are never knocked out by the flood (their fragility probability is set to 0).

Dependency links: 7,513. These are the links the simulator propagates failures along. Every site has **exactly one** supplier of each kind it depends on: the nearest one within 5 km.

link	count	median buffer (hours)	meaning
power → telecom	4,081	6	battery
fuel → telecom	1,235	48	generator fuel
power → fuel (gas station)	1,154	0	pumps need power
power → subway	485	0	traction power
fuel → water	135	48	
power → water	131	3	
telecom → power	111	2	SCADA monitoring — the only way a substation fails other than by flooding
power → hospital	60	96	generator
water → hospital	60	24	
fuel → hospital	61	96	

The **GNN's graph is larger**: 37,355 links in 20 relation types. Besides the dependency links it includes structural links (power lines, subway lines), telecom failover links and repair-access links. The simulator ignores these (repair-access is logistics, not failure). The network may still use them as message routes, because they carry information about where things are connected.

> **If asked "why nearest within 5 km?"** It is a stated engineering assumption from graph construction. No utility publishes its real feeder topology; the 5 km cap stops unrealistic cross-borough links. It is one of the assumptions listed in the report's assumptions table.

3. The floods — where the maps come from

We have **70 storms in the experiments**: 19 old GISSR + 3 GeoClaw + 48 new GISSR.

3.1 GISSR (67 of the 70)

GISSR is a fast coastal flood model. It is the code in `external/gissr_tools/gissr_core.py`, a faithful re-implementation of the lab's GISSR notebook, and it stays outside the public repo.

1. **Input:** a coastal water-level time series W(t) for one storm, taken from a synthetic storm catalogue organised by timeline, season and row.
2. **The coast** is cut into **3,199 segments of 100 m**, grouped into **206 divisions**.
3. **Inflow.** At each time step, water flows over every segment whose ground elevation e is below the water level. It is computed with Manning's equation: overtopping depth h = W − e, velocity v = (L·h/(L + 2h))^(2/3) · √S / n, inflow dV = h · L · v · dt. Here S is the slope and n the roughness of the division.
4. **Storage.** Inflow volumes are summed per division and pooled over neighbouring divisions ("redistribution radius 2"). They are then converted into a **flood height per division**, using each division's storage-volume table: how much water fills the low ground up to each height from 0 to 7 m.
5. **Depth map.** For every DEM cell in a division, depth = division height − ground elevation, wherever the ground is lower. This is `gissr_raster.write_flood_raster`: tiles are mosaicked and clipped to the borough outline; depth is in metres and 0 means dry.

The convention that matters: the delivered maps use the storm's hydrograph points **one hour apart** with all bug-fix flags off. In Week 29 this reproduced Gwen's 20 delivered maps at site level: 13 wet/dry differences among 5,827 wet sites over 20 maps. It was confirmed again today, when the new raster for t817c20 matched the depth table on all 111 wet sites.

The key fact about GISSR maps (Week 29): across 593 sampled storms, the rank correlation between peak water level and the number of wet sites is **0.9998**. A bigger GISSR storm is the smaller one plus more water. The maps are **nested by water level**: they vary in height and timing, not in spatial pattern. This one fact explains the flat learning curve of Week 32 and the motivation for map diversity.

3.2 The 20 old GISSR maps and the 48 new ones

- **Old:** Gwen delivered 20 maps. `syn_ts_810_14` is a node-level copy of `173_2` (identical to 0.35 mm), so it is held out as a determinism control. That leaves **19** in the model's set.
- **New (Week 29), selected before any cascade result:**
  - **40 training storms** with target levels evenly spaced from 1.935 to 4.5675 m. The old training storms stopped at 3.64 m.
  - They alternate **fast** storms (median site wet 2 h before the peak) and **slow** ones (12 h). This cut the correlation between level and lead time from 0.83 (in the 20 delivered maps) to 0.07, so level and timing can be told apart.
  - **8 held-out storms:** 7 midway between training levels, 1 above the range (4.72 m).
  - Each storm's catalogue coordinates (timeline, season, row) are in `data/flood/gissr48_manifest_v1.csv`.

3.3 GeoClaw (3 of the 70)

GeoClaw is a hydrodynamic shallow-water model. Its three Sandy-like runs (2026, 2050, 2080, differing in sea level) give water-surface rasters `fgout_eta_<year>_360.tif` (NAVD88). Depth = eta − USGS 1/3 arc-second ground DEM, computed at the sites (`src/flood/flood_overlay_geoclaw.py`). For timing, all three use the NOAA Battery gauge (station 8518750) hourly record from Sandy 2012: peak at 25 h, record 47 h long.

Why GeoClaw matters: it is the only flood source in our set **with a different spatial shape** from the GISSR family. It is our only test of generalisation to a differently-shaped flood.

4. Preprocessing — turning maps into per-site numbers

4.1 Depth at each site

For every storm we need one number per site: the **peak depth d** in metres.

- Old GISSR maps: the delivered raster sampled at each site's coordinates.
- GeoClaw: eta minus ground at each site.
- New GISSR: division heights → depth at sites, with the same mechanics as the raster (the raster renderer reproduces it).
- Stored as `data/flood/jesse22_node_depths_v1.csv` (23 storm columns including the control) and `gissr48_node_depths_v1.csv` (48 columns). One row per site, column `flood_<storm>_depth_m`.

Known quirk: `telecom_cluster_03042` is 0.105 m deeper in the repository table than in the depths the frozen campaign actually sampled. It was found by the Week 29 identity gate, is on record, and shows up again in the Week 32 fingerprint: its seed frequency sits 0.03–0.10 below φ in five small storms. It affects one site.

4.2 Timing at each site

For the time-variant (arrival) label mode we need *when* water reaches each site. The rule (`hydrograph_timing.py`) is the **bathtub offset**. A site with peak depth d is wet at time t when the coastal water level is within d of its peak:

> depth_i(t) = max(d_i − (W_peak − W(t)), 0)

From that curve, per site:

- **arrival_h**: first hour with depth > 0;
- **recede_h**: last hour with depth > 0;
- **duration_h**: recede_h − arrival_h;
- **time_to_peak_h**: t_peak − arrival_h, which is ≥ 0 for every wet site in all 71 storms.

All are on the storm's own clock (hour 0 = first hydrograph sample). They are stored in `node_timing_jesse22_plus_gissr48_v1.csv` (27,345 rows), with a per-storm summary (t_peak_h, span_h, wet-site count) in `_summary.csv`.

Worked example. In t198w16 (4.72 m, peak at 39.3 h), `power_bruckner` sits under 2.51 m of water at the peak. It is wet when W(t) > W_peak − 2.51, which first happens at 12.6 h, so it goes under 26.7 h before the peak. `power_east_64th_street` (1.44 m peak depth) is reached at 33.7 h, 5.6 h before the peak. Deep sites flood early, shallow sites flood late. That is why arrival labels matter.

PART II — THE SIMULATOR (WORLD 1)

5. One Monte Carlo run, step by step

Step 1 — fragility: which sites the water knocks out

For every wet, non-external site, the probability that the flood knocks it out directly is the lognormal fragility curve:

> **φ_i = Φ((ln d_i − ln δ_a) / β_a)**

Φ is the standard normal cumulative distribution. δ_a is the **median** depth at which half of sites of type a fail. β_a is the **dispersion**: how gradually the curve rises.

type	δ (m)	β
power	0.6	0.5

type	δ (m)	β
telecom	0.3	0.6
hospital	0.6	0.5
subway	0.1	0.4
water	0.5	0.5
fuel	0.5	0.6

Then, in each run, a random number decides each site: the site is a **seed** (knocked out directly) if rand < φ_i.

Worked examples.

- **Telecom at 0.222 m:** z = (ln 0.222 − ln 0.3)/0.6 = (−1.505 + 1.204)/0.6 = −0.502, so φ = 0.308.
- **Power at 0.6 m:** z = 0, so φ = 0.5.
- **Subway at 1.0 m:** z = (0 + 2.303)/0.4 = 5.76, so φ = 0.999999996. In float32 that is exactly 1.0, and it is why gate G3 needed a tolerance in Week 32.

> **Provenance, say it honestly.** The code calls these "HAZUS-FL lifeline parameters". The research could not find these six (median, β) pairs in the Hazus flood technical manual. Hazus's utility functions are deterministic depth–damage curves from USACE (1985). Treat the values as **project assumptions in the Hazus style** until a source table is found. This matters for the end-to-end discussion: what we would "learn" is our assumed curve, not a physical truth.

> **Why only peak depth?** Fragility in World 1 uses the peak depth only, not duration or velocity. That is a modelling choice. Hazus works the same way, and the literature finds depth the dominant predictor (Wagenaar et al. 2017: velocity and duration had importances of about 0.04–0.05).

Step 2 — buffers: how long a site survives without its supplier

Every dependency link has a buffer: how long the downstream site keeps working after its supplier fails. In each run every buffer is drawn from a **Weibull distribution** whose median is the engineering value in the table above (6 h battery, 96 h generator, and so on). The shape depends on the receiving type: hospital 2.5, telecom 3.0, water 2.5, subway 4.0, fuel 2.0, power 3.0. A shape above 1 means "failure gets likelier the longer you wait". Zero buffers stay zero.

Step 3 — the cascade (`src/simulation/cascade_joint.py`)

The engine looks at the city on a time grid. At each look t, it repeats two moves until neither adds a failure:

- **(a) dependency rule — OR gate.** A working site fails at t if **any one** of its suppliers failed at time T and T + buffer ≤ t. One supplier per kind, so a telecom cluster dies when its substation has been out longer than its battery, *or* its fuel source longer than 48 h, whichever comes first.
- **(b) within-system rules**, applied instantly to the current set of failed sites:
  - **subway:** Motter–Lai overload. Each station's capacity is (1 + α) × its normal load with α = 0.2. When stations die, load redistributes, and stations pushed over capacity fail.
  - **telecom:** each cluster's capacity is L0/ρ with ρ = 0.6. After the peak, demand surges by μ(t) = 1 + 0.5·e^(−t/12). A cluster crashes above 3× capacity, and spill-over reaches 2 hops within 3 km.
  - **hospital:** gravity routing of patients. Baseline census 0.75, diversion at 0.95, collapse at 1.20 × beds, distance scale 5 km, reach 40 km. Anchored to Sandy (Lee et al. 2015).
  - **water and fuel:** loss of reachability within their networks.
  - **power:** none. There is no power-flow physics in these runs; Jesse's pandapower coupling exists but is switched off.

A failure caused by a zero-buffer link lands at the *next* look, never retroactively at hour 0. This was the "t = 0 semantic fix" of v2: hour 0 contains only the flood seeds.

Step 4 — two label modes, two clocks

	static	arrival (time-variant)
when a seed fails	at the peak (the record's hour 0)	at its own `arrival_h`
grid	0, 6, 24, 48, 96 h	every 6 h from onset to the peak, then peak + 0, 6, 24, 48, 96, 144, 240, 360
record frame	hours after the peak	hours after the peak (negative = before)
seed decision	peak depth	peak depth (same)

Why both? Static is the simple, well-understood baseline. Arrival is more realistic: sites go down in the order the water reaches them, and the cascade starts while the storm is still rising. Arrival labels give 2–50 % more failures than static (Week 29). For geoclaw_2080 the mean failed count by 96 h is 2,509 under static and 3,681 under arrival.

Correction on record (Week 32). The static campaigns never stored the seed set (`initial_failures`); the arrival campaigns store it in every run. The code reconstructs static seeds as "the sites stamped at hour 0". The fingerprint proved this exact: seed counts and every per-site seed frequency are identical between the two modes, for example 206.6 seeds for geoclaw_2026 in both.

Step 5 — 1,000 runs per storm

Each storm is simulated 1,000 times. Seeds and buffers are redrawn each run; the map and graph are fixed. Campaigns on the Torch cluster:

- `legacy_v1_n1000` (old storms, static) and `dynamic_v1/arrival_h360` (old storms, arrival);
- `gissr48_v1/static` and `/arrival_h360` (new storms).

Before production, each campaign had an identity gate: re-running an old storm through the new wrapper must reproduce the frozen records exactly.

Why 1,000? A per-site probability estimated from n runs has standard deviation $\sqrt{\pi(1-\pi)/n}$. At n = 1,000 that is at most 0.016 (at $\pi = 0.5$). That noise is small enough to train on, and it sets the floor in §7.3.

6. From runs to targets

For each storm, each site i and each horizon t_k (`src/gnn/data.py::marginal_labels`):

- **$\hat{\pi}_{ik}$** = share of the 1,000 runs in which site i had failed by t_k, seeds included. This is the **target** of the direct model.
- **s_i** = share of runs in which site i was a seed. It is the Monte Carlo estimate of φ_i, used by the fingerprint check F3 and the key to the end-to-end design.
- **halves**: $\hat{\pi}$ from runs 1–500 and from runs 501–1,000, used for the noise floor.

These were computed once per label mode and cached (`$SCRATCH/results/gnn_direct_targets_v1/{static,arrival}` 70 files each). Training reads only the cache.

The identity you must know. By the law of total probability:

$$\hat{\pi}_{ik} = s_i + (1 - s_i) \cdot \hat{g}_{ik}, \text{ so } \hat{g}_{ik} = (\hat{\pi}_{ik} - s_i) / (1 - s_i)$$

$\hat{g}$ is the share of the *non-seed* runs in which the site failed by t_k: the **pure cascade probability**. This holds exactly under static labels, because seeds fail at hour 0 and are counted in $\hat{\pi}$. It also holds under arrival labels in our data, because every seed arrives before the peak and so before every horizon. This identity is what makes the end-to-end model identifiable (Chapter 12).

Check (F2/F3, Week 32):

- An independent recount from the raw records equals the cached targets on 70/70 storms in both modes (F2).
- Seed frequency is within 0.03 of φ on 99.08 % of 26,976 wet site-storms (F3). The 0.9 % outside is binomial noise (244 observed, 237.8 expected), plus the 03042 quirk.

PART III — THE FAST MODELS (WORLD 2)

7. Why a surrogate, and how we score one

The simulator takes minutes to half an hour per storm for 1,000 runs. We want answers for thousands of hypothetical storms in seconds: for planning, for screening, and eventually for real-time use. So we train a neural network (the **surrogate**) to reproduce the simulator's answers.

7.1 The error score: soft log loss

For one site and horizon, the model says p and the simulator says $\hat{\pi}$:

$$L = -[\, \hat{\pi} \ln p + (1 - \hat{\pi}) \ln(1 - p) \,]$$

This is averaged over 6,231 sites × 4 horizons, then over storms.

- It is a **proper scoring rule**: its expectation is minimised only at $p = \hat{\pi}$.
- Its minimum value is not zero. It is the **entropy** $H(\hat{\pi}) = -\hat{\pi} \ln \hat{\pi} - (1 - \hat{\pi}) \ln(1 - \hat{\pi})$. A site that fails in half the runs cannot be predicted with certainty.

Worked example, $\hat{\pi} = 0.3$:

- p = 0.3 gives 0.611 (the best possible);
- p = 0.4 gives 0.633;
- p = 0.03 gives 1.073. Confident and wrong is punished hardest.

7.2 Other scores

- **Count error:** $(\Sigma_i p_i - \Sigma_i \hat{\pi}_i) / \Sigma_i \hat{\pi}_i$, the relative error in the expected number of failed sites.
- **Calibration (ECE):** among sites predicted at about 30 %, do about 30 % fail? Measured over ten bins.
- **Ranking:** average precision against $\hat{\pi} > 0.5$, and the Spearman rank correlation.

7.3 The floor

Score $\hat{\pi}$ from runs 1–500 against $\hat{\pi}$ from runs 501–1,000 (and the reverse, averaged). This is the loss a predictor would get **even if it knew the simulator's true answer**, because the target itself is a 1,000-roll estimate. It is about the entropy plus 0.001 per uncertain site. Static 13-storm mean floor: **0.064**; arrival: **0.084**.

8. The two-stage model (Weeks 19–31)

The two-stage model learns **one run at a time**. Its inputs per site are:

- the 8 attributes;
- a **seed bit** (was this site knocked out in this run);
- a storm-size summary built from the run's seed count, $c_a = \ln(1 + k_a)/\ln(1 + N_a)$ per type and overall;
- optionally the timing columns.

It is trained against each run's 0/1 outcome (BCE). To predict for a new map: draw 100 seed sets from φ, run the network once per set, and average.

Last results (Week 31, 57 training storms with the storm summary): per-run non-seed BCE 0.054 / 0.054 / 0.109 / 0.099 in the four cells, ahead of the nearest-storm lookup on 12 of 13 storms in every cell.

> **Why 0.054 last week and 0.071 this week are both right.** 0.054 is the error *when the model is told which sites the water took out*. The storm-level score, from the map alone with the draws averaged, is 0.071 for the same model. More uncertainty is left, so the number is higher.

9. The direct model (Week 32) — every step, with calculations

9.1 Inputs per site — width 17 (23 with timing)

columns	what	example: Coney Island–Stillwell Av station in 914_6
8	static attributes (lat, lon, two constant placeholders, four type-specific; each scaled to [0, 1], constants become 0)	—
1	depth: min(d, 5 m)/5	0.203 m → 0.0407
1	φ from the formula	0.962
7	expected-seed summary c_a = ln(1 + Σφ_a)/ln(1 + N_a), six types + overall; identical for every site in the storm	overall: ln(92.3)/ln(6,232) = 4.525/8.737 = 0.518
(3)	timing /96 h: arrival, duration, time-to-peak (timing cells only, plus 3 timing means added to the summary)	7.8, 2.6, 0.9 h → 0.081, 0.027, 0.010

Why each piece:

- **Depth is capped at 5 m** because every fragility curve is saturated far below 5 m; deeper water carries no extra information and would only stretch the input range.
- **φ is given** because it is exactly what the simulator draws seeds from.
- **The summary exists** because two message-passing layers see only sites within two links. A dry site far from the water would otherwise get the same input in every storm and could not tell a small storm from a large one (the Week 29 finding).
- **Expected rather than realised seed count** because there is no draw in the direct model.

9.2 Inside the network (`src/gnn/model.py::CascadeGNN`)

1. Per-type projection. Each type has its own linear map: $h^0_i = \mathrm{ReLU}(W_a x_i + b_a)$, from 17 inputs to 64 numbers.

- Parameters: 6 types × (17 × 64 + 64) = **6,912**.
- *Why per-type:* a hospital's attributes mean something different from a cell cluster's.

2. Message passing — two layers of relation-wise attention (TransformerConv inside HeteroConv). For each of the 20 relation types r, and each site i with neighbours j along r:

```
query  q_i = W_q h_i        key  k_j = W_k h_j        value  v_j = W_v h_j
attention  α_ij = softmax_j( q_i · k_j / √16 )      (per head; 4 heads × 16 dims = 64)
message    m_i^r = W_skip h_i + Σ_j α_ij v_j
```

- Messages from all relations are **summed**, then residual + ReLU: $h^1_i = \mathrm{ReLU}(\Sigma_r m_i^r + h^0_i)$. Repeat once more for h^2.
- Parameters per relation per layer: 4 linear maps of 64 × 64 + 64 = 16,640. Over 20 relations × 2 layers: **665,600**.
- *Why attention:* a site may have many neighbours of one relation (a substation feeds ~20 cell clusters), and attention lets the model weigh which neighbour matters.
- *Why two layers:* the receptive field is two links. A cell cluster can see its substation and that substation's SCADA link; it cannot see three hops away. The storm summary is the workaround for distance.

3. Per-type output head. Linear 64 → 64, ReLU, dropout 0.1, Linear 64 → 4, giving four numbers $\eta_1..\eta_4$ per site.

- Parameters: 6 × (64 × 64 + 64 + 64 × 4 + 4) = **26,520**.

4. Hazard read-out (Week 27). The four numbers are read as **window hazards**:

```
h_k = σ(η_k)                P(fail in window k | survived to its start)
S_k = Π_{j≤k} (1 − h_j)     P(survived to t_k)
g_k = 1 − S_k               P(failed by t_k)   − non-decreasing in k, by construction
```

- *Why:* "failed by 24 h" can never be less likely than "failed by 6 h". The independent head violated this on 4–8 % of site-horizon pairs (Week 27); the hazard head gives exactly 0 violations.

Total: 6,912 + 665,600 + 26,520 = 699,032 parameters (701,336 at width 23). You can rebuild this by hand.

9.3 The composition (outside the network)

$$p_{ik} = 1 - (1 - \varphi_{ik})(1 - g_{ik})$$

Read it as: the site has failed by t_k if the water took it out (φ), **or**, failing that, the cascade reached it (g). By the identity of §6, **g is exactly the conditional cascade probability**, so the network learns only the cascade.

Properties:

- $p \geq \varphi$;
- p rises with time;
- $\varphi = 1$ forces $p = 1$;
- $\varphi = 0$ gives $p = g$.

Example: $\varphi = 0.308$ and $g = 0.5$ give $p = 1 - 0.692 \times 0.5 = 0.654$.

In code this is done in log space for numerical stability: $\log(1 - p) = \log(1 - \varphi) - \mathrm{softplus}(\mathrm{logit}\ g)$.

9.4 Training

- One example per storm. Adam with learning rate 10^{-3} and weight decay 10^{-5}; dropout 0.1.
- Up to 600 epochs, early stopping on the 7 validation storms with patience 50; two PyTorch seeds.
- Split by storm: **50 train / 7 validation / 13 test**. The test storms are last week's held-out 13, so every comparison is like for like.

9.5 What happened (Week 32, on record)

- **One pass equals a hundred:** 13-storm means 0.069 vs two-stage 0.071 (static), 0.104 vs 0.105 (arrival + timing). The two-stage model is ahead on only 2–5 of 13 storms. **11 ms vs ~1 s per storm.**
- **Static is within 7 % of the floor** (0.069 vs 0.064); arrival is 25–40 % above it.
- **Storm size is right:** within 3 % of the simulator on all 13 storms (static).

- **The learning curve:** 17 → 34 storms gave 13 % better, all of it above 3.6 m (levels the old storms lacked); 34 → 50 gave nothing. *Diversity, not count, limits it.*
- **GeoClaw under arrival labels:** gets worse as GISSR storms are added (0.16 → 0.18).
- 3 of 7 pre-registered predictions passed.

PART IV — THE NEW APPROACH: MAKE IT END-TO-END

10. What was asked, precisely

At the last meeting you pointed out that the direct model still applies a formula, $\varphi = \Phi(\cdot)$, to each new map before the network sees it. Dr. Miura asked for the pipeline to be **end-to-end**: **depth in, failure probability out, with the fragility learned by the network rather than handed to it.**

Be exact about what changes:

step	direct model (now)	end-to-end (proposal)
read depth at each site from the map	data preparation	data preparation (unchanged; reading a map is not physics)
φ from depth	Hazus-style formula, outside	**learned fragility head, inside the network**
storm summary	from formula φ	**from learned $\hat{\varphi}$**
cascade g	GNN	GNN (unchanged)
combine	$p = 1 - (1 - \varphi)(1 - g)$	$p = 1 - (1 - \hat{\varphi})(1 - g)$ (unchanged)
arrival timing gate on φ	indicator, outside	same (timing stays out of the fragility head; §17)

11. The honest framing (say this first)

On our data, a learned fragility can at best rediscover the curve the simulator used, because the simulator decided every direct failure with exactly that curve. So the end-to-end model will not be *more accurate* on these 70 storms. What it gives:

1. **A demonstration:** the network can learn the physics step itself, from a deliberately wrong starting point, and we can measure how well. That makes the pipeline genuinely map → probability.
2. **A door Hazus cannot open:**
   - a learned fragility can take inputs the curve ignores (how long the water stays, how fast it rises, site attributes such as elevation or flood walls);
   - it can be calibrated to **real outages** (Sandy) rather than to an assumed curve;
   - and if the curve is wrong for a type, a flexible head can find out.

   This is the research value.
3. **Novelty:** the literature review found GNN infrastructure surrogates that take fragility *as an input* (e.g. Liu & Meidani 2024 for bridges, where lognormal failure probabilities are edge features). Others start from a given set of failed nodes (the I^3 cascade model). It found **none that learns flood fragility inside the network from a hazard map.**

12. The central problem: identifiability (understand this fully)

12.1 The problem in one example

A substation is under 0.6 m of water. Suppose the truth is $\varphi = 0.5$ (direct) and $g = 0.4$ (cascade):

> $p = 1 - (1 - 0.5)(1 - 0.4) = 1 - 0.5 \times 0.6 =$ **0.70**

Now suppose the network instead learns $\hat{\varphi} = 0.7$ and $\hat{g} = 0$:

> $p = 1 - 0.3 \times 1 =$ **0.70**

Same output, same loss. If we only train on "failed by t" ($\hat{\pi}$), the network cannot tell "the water did it" from "the cascade did it". Any split that gives the same total is equally good. In ML terms the decomposition is **not identifiable**.

12.2 Why it is worse than it sounds: cascade leverage

There is also a systematic push in one direction, not just ambiguity. A substation feeds about 20 cell clusters, a few water facilities, subway stations and gas stations. Making the substation *more fragile* in the model raises the failure probability of everything downstream. If the cascade part is underfitting anywhere, the cheapest way for gradient descent to fix the totals is to inflate the upstream fragility. So without supervision, expect **power and telecom to be learned as more fragile than they are.** The research report frames this exactly as an unidentified decomposition. The APHYNITY paper (Yin et al., ICLR 2021) says of physics-plus-network models F = F_physics + F_network that the split "is in general not unique… this hampers the interpretability".

12.3 The fix we have and most papers do not

We **observe the direct part separately**: s_i, the seed frequency, is known for every wet site in every storm. So we add a second target:

> **L_total = SoftBCE(p, $\hat{\pi}$) + λ · SoftBCE($\hat{\varphi}$, s)** (the second term over wet, non-external sites)

The second term pins $\hat{\varphi}$ to what the water actually did. Then g must explain the rest, which is exactly $\hat{g} = (\hat{\pi} - s)/(1 - s)$. The decomposition becomes identifiable. Optionally add a third term $\lambda_g \cdot$ SoftBCE(g, $\hat{g}$) to supervise the cascade part directly.

Why not the APHYNITY fix (make the network part as small as possible)? Here the cascade is real, not a correction term. Penalising g would push the model to explain cascade failures as direct ones, inflating $\hat{\varphi}$, which is the very bias we want to avoid. We have direct supervision of the physics term, so we use it.

12.4 Design principle to defend

> "If a quantity is meant to be physical, supervise it with its physical observation. Do not hope the end task will make it come out right."

13. Where the information about fragility actually is

The fragility parameters are learned from sites whose φ is neither near 0 nor near 1. The gradient shows why. With $z = (\ln d - m)/\beta$, where $m = \ln \delta$:

> $\partial\varphi/\partial m = -\phi(z)/\beta$ $\partial\varphi/\partial\beta = -\phi(z) \cdot z/\beta$ (ϕ = normal density)

power site at depth	φ	∂φ/∂m	∂φ/∂β
0.15 m	0.003	−0.017	+0.047
0.30 m	0.083	−0.305	+0.423
0.60 m (median)	0.500	**−0.798**	0.000
1.20 m	0.917	−0.305	−0.423
2.40 m	0.997	−0.017	−0.047

Three lessons:

1. **Deep sites teach nothing.** At 2.4 m the curve is flat, so moving the median barely changes φ. GeoClaw depths saturate the curves, which is why the Week 11 surge-scaling sweep (×1.0 to ×1.5) changed expected failures by less than 0.2 %.
2. **The median is learned from sites near the median**, and the **spread from sites on the shoulders** (around ±1 β from the median). At the median itself the β-gradient is zero.
3. **Count how much information each type has** (50 training storms, wet non-external site-storms):

type	wet site-storms	informative (0.05 < φ < 0.95)	saturated (φ > 0.99)	median depth
telecom	13,193	4,876	5,264	0.96 m
fuel	2,386	1,380	280	0.81 m
water	1,169	554	278	0.93 m
power	551	312	54	0.83 m
hospital	187	**105**	16	0.78 m
subway	1,077	**114**	875	0.80 m

Each informative site-storm carries 1,000 Bernoulli draws, so even 105 of them pin a 2-parameter curve fairly well. But **subway and hospital will be recovered least precisely**: subway because its median is only 0.1 m and nearly every wet station is saturated, hospital because there are few hospitals. Write that into the predictions before training.

14. The fragility head — choices and the recommended one

14.1 Parametric lognormal head (recommended main arm)

```
per type a:  m_a (learnable, = ln δ_a)    b_a (learnable)    β_a = softplus(b_a)  (or exp)
φ̂i = Φ( (ln max(d_i, ε) − m_a) / β_a )   for wet non-external sites, 0 otherwise
```

- 12 parameters in total, directly comparable to the table in §5.
- Monotone in depth by construction.
- Differentiable end to end, so gradients flow from the loss into m_a and b_a.

Why parametric first: the truth in World 1 *is* lognormal, so a correctly specified head is the cleanest test. The fragility literature (Lallemant et al. 2015; Bayesian work such as Van Biesbroeck et al. 2025) also recommends the parametric probit-lognormal whenever data are few and binary, which real outage data always are.

Note on precedent: the repository already has this layer, on the branch `week11-learnable-fragility` (`src/gnn/learnable_fragility.py`, log-space median and β, with a penalty pulling back towards Hazus). Its only run started **exactly at Hazus**, with the penalty on and labels from the leaky v1 era, and trained 10 epochs. Its parameters moved by less than 0.2 % (e.g. power median 0.6008, β 0.4999). **That run proves nothing**: it started at the answer, was told to stay there, and had no pressure to move. Our design fixes all three problems: perturbed start, no penalty towards Hazus, honest targets plus seed supervision.

14.2 Flexible monotone heads (comparison arms)

- **Mixture of K lognormals per type** (softmax weights, positive βs). Still a monotone curve, and it can represent step-like behaviour such as Con Edison's "fails once the design flood level is exceeded". With K = 1 it is the parametric head.
- **Cumulative hazard on a depth grid:** φ(d) = 1 − exp(−Σ_{knots ≤ d} softplus(θ_k)). This is the same trick as our hazard read-out over time, applied over depth. It is monotone and non-parametric, but constant beyond the last knot, which is a risk for deep GeoClaw water.
- **Constrained monotone neural network** (Runje & Shankaranarayana, ICML 2023). It is monotone in chosen inputs (depth, and later duration and velocity) and unconstrained in others (site attributes). This is the tool for the multi-feature future.

14.3 Black box (comparison arm)

No head and no composition. Depth is just a feature and the network outputs p directly. This shows what structure buys. Expect it to fit the training storms and to be worse on GeoClaw and small storms, with some sites where more water *lowers* p (a monotonicity violation).

15. Your "stacking" idea — named properly, and the three ways to train it

Name: this is not *stacking*. Stacking (Wolpert 1992) means training several models on the same task and a meta-model that combines their predictions. Your idea is a **two-stage modular model with an interpretable intermediate quantity**: model 1 maps depth to φ, model 2 maps φ to the cascade. The ML literature calls this a **concept bottleneck model** (Koh et al., ICML 2020), and φ is the "concept".

The three training regimes (Koh et al.), as they apply here:

regime	what it means here	strength	risk
independent	fit model 1 on (depth, s) alone. This is just a probit regression per type and takes seconds. Train model 2 on the *true* φ (= our current direct model).	φ is purely physical; you can swap in a better fragility later without retraining	model 2 never sees model 1's small errors
sequential	fit model 1 first, freeze it, train model 2 on model 1's *predicted* $\hat{\phi}$	model 2 adapts to model 1's errors; φ stays physical	none here
joint	one network, L = L_task + λ·L_concept, trained together	slightly better task accuracy in Koh et al.	**leakage:** the cascade loss can bend $\hat{\phi}$ to carry cascade information (Margeloiu 2021, Mahinpei 2021)

Why joint should be safe here, as a testable claim. Havasi et al. (NeurIPS 2022) trace leakage to two causes: an incomplete concept set, and a concept predictor too weak for the true concept. Neither applies here. φ is the *only* quantity the simulator passes from depth to the cascade, so the set is complete; and the head has the true functional form, so it is correctly specified. Prediction: all three regimes land within about 1 % of each other. If so, prefer **sequential**: it is as accurate, and φ stays interpretable and replaceable.

16. Leakage, and the two plugs

There are two routes by which the old formula, or cascade signal, could leak into the "learned" fragility:

1. **The storm summary.** Today it is computed from formula φ (`storm_context_expected`). If the end-to-end model still computes it from the formula, the formula has never left the pipeline. **Fix: compute Σ$\hat{\phi}$ from the learned head.** In the main arm, stop the gradient through the summary, so the summary cannot become a back door for tuning $\hat{\phi}$.
2. **$\hat{\phi}$ as a network input.** If $\hat{\phi}$ feeds the GNN as a feature *and* gradients flow back through that feature, the cascade loss can shape $\hat{\phi}$ for cascade purposes. **Fix: detach $\hat{\phi}$ where it enters the GNN as a feature.** The fragility parameters then learn only from the seed loss and the composition. Run one arm without the detach to measure whether it matters.

17. Timing: keep it out of the fragility head

In World 1, *whether* a site is knocked out depends only on peak depth; *when* depends on arrival. So:

- the fragility head takes depth only;
- the time gate stays outside: $p_{ik} = 1 - (1 - \hat{\phi}_i \cdot A_{ik})(1 - g_{ik})$, with $A_{ik} = 1$ if the site's water has arrived by t_k. In our storms that is always 1, since every wet site arrives before the peak.

A good null test: give the head duration and arrival as extra monotone inputs and check that their learned coefficients come out ≈ 0, because World 1 does not use them. If they do not, the head is picking up cascade signal, which would be a leak.

Physical caveat: using peak depth to decide a failure that happens at arrival time is non-causal (the site "knows" its future peak). The simulator does this. A future real-world version should condition on depth-to-date.

18. The experiment that can actually show learning beats the formula: World 1b

Everything above can at best recover the curve. To show that learning the fragility *adds value*, the true fragility must differ from the assumed one. **World 1b** is a small extra simulation batch, 10–20 storms, where the simulator's direct failures follow a curve that is *not* the assumed lognormal. For example:

- a step-like curve (a substation is fine until the water passes its design level, then fails);
- or a curve that also depends on duration.

Then compare:

- **(a)** the direct model with the old formula (now misspecified);
- **(b)** the parametric learned head;
- **(c)** the flexible monotone head.

Prediction: (c) beats (b), and (b) beats (a), on both curve error and total loss.

This requires **new simulation**, so it is an advisor decision. It is also the experiment that makes the end-to-end idea a contribution rather than a demonstration.

19. Further ahead: calibrating to Sandy

The research collected candidate *direct-failure* observations from Sandy 2012. These must be verified against primary sources before use:

- **power:** Con Edison's East River, East 13th Street and Seaport substations flooded; the 11 networks lost were *cascade* outcomes;
- **hospitals:** NYU Langone, Bellevue and Coney Island were evacuated after electrical and mechanical failures;
- **subway:** eight tubes under the East River and Newtown Creek flooded;
- **telecom:** Verizon's 140 West St central office had its basement cable vault flooded.

Rules if we ever do it:

- **Use direct failures only.** Total outage counts mix direct and cascade failures; that is the identifiability problem again, with real data.
- **Bayesian update of the 12 parameters**, with a prior centred on the current values. A neural head cannot be trained on a few dozen binary observations.
- **Beware:**
  - modelled depths are uncertain, which biases the learned β upwards (Ader et al. 2018);
  - GeoClaw saturates the curves at the sites that failed, so a Sandy pattern alone may not constrain the parameters (the Week 11 sweep already showed this).

20. Ensembles (the other meaning of your idea)

A **deep ensemble** (Lakshminarayanan et al. 2017) trains the same model 5 times from different random starts and averages the outputs. Our output is already a probability, so it already contains the simulator's randomness. What an ensemble adds is the **model's own uncertainty**: where the five members disagree, the model does not know.

Prediction: the five members agree almost perfectly on the 12 fragility parameters but disagree on the cascade part g, most of all on GeoClaw. That makes ensemble spread a useful **"this storm is unlike my training data" flag**. It is cheap (5× training) and optional.

PART V — THE PLAN

21. Evaluation arms

arm	fragility	training	question it answers
A0	formula given (= Week 32 direct model)	soft BCE	baseline
R	probit regression of s on ln d per type	maximum likelihood, seconds	the best achievable parameter recovery (oracle)
A1a	learnable lognormal, **perturbed start** (median ×2 or ×0.5, β ×2 or ×0.5)	total + seed loss	**main design**
A1b	same as A1a	total loss only	identifiability demonstration
A2	flexible monotone head (mixture or depth-grid cumulative hazard)	total + seed loss	what flexibility costs and buys
A3	none (black box, depth as feature)	total loss	what structure buys
A4	two-stage: independent / sequential / joint ($\lambda \in \{0.1, 1, 10\}$)	per Koh et al.	modular vs joint
A5 (optional)	5-member ensemble of the best arm		uncertainty
M (needs new sim)	A1a vs A2 on World 1b		where learning beats the formula

Same split (50/7/13), same four cells (or start with static_plain and arrival_timing only, to halve the cost), two seeds.

22. Metrics

1. **Parameter recovery:** learned median and β vs the assumed values, per type, with a bootstrap interval over storms.
2. **Curve error:** mean $|\hat{\varphi} - \varphi|$ over the real wet-depth distribution of each type, and the seed-frequency log loss against its binomial floor.
3. **Total soft log loss:** its excess over the floor, and its ratio to A0, per horizon.
4. **Count error** per storm.

5. **Decomposition error:** mean $|g - \hat{g}|$ on wet sites. Only an identifiable model gets this right.
6. **Monotonicity violations:** share of site-storms where +10 % depth lowers p.
7. **Calibration** (ECE), and for A5, how well spread tracks error.

Strata: all 13 test storms; GeoClaw (the shape-shift test); the smallest storms (where curve tails matter).

23. Draft predictions (to freeze before training, exactly like Week 32)

- **P1** — A1a recovers the medians within 3 % and β within 5 % for power, telecom, water and fuel. Subway and hospital are within 10 %, because they have the fewest informative sites (§13).
- **P2** — A1b fits the *total* loss within 2 % of A0, but at least one of power or telecom ends ≥ 10 % off in median, biased towards *more fragile*. This is the identifiability signature.
- **P3** — A1a's total loss is within ±2 % of A0's.
- **P4** — A2 matches A1a on the test storms but has larger curve error outside the training depth range and does worse on GeoClaw.
- **P5** — A3 is worse than A0 on GeoClaw and small storms and has a non-zero monotonicity-violation rate.
- **P6** — the three A4 regimes differ by less than 1 % in total loss.
- **P7** — extra duration and arrival inputs to the head come out ≈ 0 (null test).
- **P8** — on World 1b (if run), A2 beats A1a, and A1a beats the formula.

24. Implementation map (when we build it)

file	change
`src/gnn/model.py`	`FragilityHead` (parametric; flexible variant), with initialisation options; the composition is unchanged
`src/gnn/data.py`	the direct example passes depth and external mask; the storm summary is computed inside the model from $\hat{\varphi}$ (detached)
`src/gnn/train.py`	seed loss with weight λ (target s from the cache; it is already there), optional $\hat{g}$ loss, regimes (independent / sequential / joint), learning rate for the head, perturbed initialisation
`scripts/week33_fragility_fingerprint_v1.py`	the default build is unchanged (as F1); the parametric head initialised at the assumed values reproduces formula φ bit for bit; the probit regression (R) on the cached s recovers the parameters; gradients reach m and b; detach checks
`scripts/week33_fragility_audit_v1.py`	parameter recovery, curve error, decomposition error, monotonicity, plus everything of Week 32
`scripts/week33_fragility_table_v1.py`	the frozen P1–P8

Cost: no new simulation except World 1b. The targets and the seed frequencies are already cached. Training takes 5–25 min per run, as in Week 32.

Order (cheap before expensive):

1. Oracle R on CPU, in seconds.
2. Fingerprint.
3. Freeze the predictions.
4. Smoke run.
5. Arms A0 / A1a / A1b first; then A2 / A3 / A4.

6. World 1b only if the advisors want it.

PART VI — DEFENDING IT

25. Questions and answers

“Why not let the network learn the fragility from the start?” “Now we do, but as a layer with the right shape, supervised by what the water actually did. Learning it without that supervision gives a model that fits the totals and gets the physics wrong; I can show that with arm A1b.”

“If the simulator used Hazus, isn’t this circular?” “On World 1, yes: the best it can do is rediscover the curve, and I say so. The value is that the same head can then use inputs the curve cannot, or be fitted to real outages, or be tested on a world where the curve is wrong (World 1b). Recovering the known curve from a wrong start is the validation that the machinery works.”

“How do you know the network learns fragility and not cascade?” “Two ways. The seed-frequency loss supervises the direct part separately. And the decomposition error $|g - \hat{g}|$ measures whether the cascade part is right on its own, because we know $\hat{g} = (\hat{\pi} - s)/(1 - s)$ exactly.”

“Why a lognormal and not a free neural curve?” “Because the true curve in World 1 is log-normal, and the fragility literature prefers the parametric form when data are scarce and binary, which real outage data are. The free monotone curve is a comparison arm: if the lognormal were too restrictive, that arm would show it.”

“Is this end-to-end if you still fix the shape?” “Yes, in the sense that matters: depth goes in, and no external formula is applied. The network carries the fragility parameters and learns them from data. Fixing the shape is a modelling choice, like choosing a sigmoid output layer, and arm A2 tests it.”

“Why not train model 1 and model 2 separately — your stacking idea?” “That is the ‘independent’ or ‘sequential’ concept-bottleneck regime, and it is in the plan. I expect it to be as good as joint training here, because model 1 is a correctly specified 12-parameter curve. If so, sequential is preferable: the fragility stays interpretable and can be swapped out.”

“Why is that not stacking?” “Stacking combines several models’ predictions for the same task. Here the models do different tasks in sequence. It is a modular, or concept-bottleneck, model.”

“Which sites tell the model about fragility?” “Only sites whose water depth is near the curve’s rising part. Deep sites are saturated and carry no gradient. For subway that is only 114 site-storms, because the median is 0.1 m, so subway will be the least precisely recovered.”

“Why perturb the starting point?” “Starting at the answer cannot test recovery. The Week 11 run started at Hazus, with a penalty keeping it there, and its parameters moved by less than 0.2 %. That shows nothing either way.”

“Could the storm summary smuggle the formula back in?” “Yes, if it were computed from the formula φ. In the new design it is computed from the learned $\hat{\phi}$, and its gradient is stopped.”

“What about arrival times?” “In the simulator, whether a site is knocked out depends on peak depth only; when depends on arrival. So timing stays outside the fragility head as a gate. As a null test, giving the head duration should produce a coefficient near zero.”

“Will this fix GeoClaw?” “No. GeoClaw’s problem is the cascade under a different spatial shape, not fragility: GeoClaw depths are mostly deep and saturated. Fixing it needs differently shaped training maps.”

“What would real data add?” “A fragility calibrated to what actually failed in Sandy, using direct failures only, through a Bayesian update of the 12 parameters from the current values. With a few dozen observations it is a correction, not a re-learning.”

“How many parameters does the new part add?” “Twelve for the parametric head. Everything else is the same 699,032-parameter network.”

“What exactly is $\hat{g}$ and is it exact?” “The share of runs in which the site failed by t_k among runs where it was not a seed. $\hat{\pi} = s + (1 - s)\cdot\hat{g}$ by total probability. It is exact under static labels and, in our storms, under arrival labels too, because every seed arrives before the first horizon.”

“Where did the six fragility parameter pairs come from?” “They are the project’s Hazus-style assumptions. The Hazus flood manual’s utility functions are depth–damage curves from USACE (1985). I have not found these exact lognormal pairs in the manual, so I cite them as assumptions. That is one more reason a learnable head is worth building.”

26. Glossary

term	meaning
site / node	one piece of infrastructure in the graph (6,231)
seed	a site the water knocks out directly in a run
φ (phi)	probability that the water knocks a site out (fragility), from depth
s	share of 1,000 runs in which the site was a seed (estimate of φ)
$\hat{\pi}$ (pi-hat)	share of 1,000 runs in which the site had failed by t_k (the target)
g, $\hat{g}$	cascade probability given not a seed (model / from data)
static / arrival	label modes: all seeds at the peak / each seed at its water’s arrival
buffer	hours a site survives after its supplier fails (Weibull per run)
floor	the loss a perfect predictor still pays, from the target’s own noise
soft log loss	the training and scoring error against a probability target
hazard read-out	network output as per-window failure rates, chained into a rising curve
composition	$p = 1 - (1 - \varphi)(1 - g)$
identifiable	only one split of the explanation fits the data
concept bottleneck	model with an interpretable middle quantity (here φ) between input and output
leakage	the middle quantity secretly carrying information it should not
World 1 / 1b	the simulator / a variant with a deliberately different fragility

27. Numbers to know

- **6,231 sites** (203 / 4,150 / 61 / 493 / 137 / 1,187); **99 external**; **7,513 dependency links**, one supplier per kind, OR gate; GNN graph 37,355 links in 20 relations.
- **70 storms** = 19 old GISSR + 3 GeoClaw + 48 new GISSR. GISSR: 206 divisions, 3,199 segments; level–wet correlation **0.9998**.
- **1,000 runs** per storm per label mode; target noise ≤ **0.016**.
- Fragility: power 0.6 / 0.5, telecom 0.3 / 0.6, hospital 0.6 / 0.5, subway 0.1 / 0.4, water 0.5 / 0.5, fuel 0.5 / 0.6. Half of wet site-storms have $\varphi > 0.96$; **19 %** lie between 0.2 and 0.8.
- Direct model: width **17 / 23**; **699,032 / 701,336** parameters (6,912 + 665,600 + 26,520).
- Week 32: **0.069 vs 0.071** (static), **0.104 vs 0.105** (arrival + timing); floor **0.064 / 0.084**; **11 ms vs 1 s**; storm size within **3 %** on 13/13; curve flat after **34**; 3 of 7 predictions passed.

- End-to-end head: **12 parameters**. Informative site-storms: telecom 4,876, fuel 1,380, water 554, power 312, subway **114**, hospital **105**.
- Seed check: **99.08 %** within 0.03; **244** outside vs **237.8** expected by chance.